Alocação Sequencial - Filas

Estruturas de Dados e Introdução à Projeto e Análise de Algoritmos

Prof. Luis Souza

Fila

- Conceito intuitivo: fila de pessoas
  - Inserções de novas pessoas no final da fila
  - Remoções de pessoas do começo da fila
- Definição trivial:
  - FIFO: First In First Out
- Exemplos de programas que utilizam fila:
  - Impressão de documentos.

Operação	Retirado	Fila
F		F
I		FI
L		FIL
.	F	IL
A		ILA
.	I	LA
E		LAE
X		LAEX
.	L	AEX
.	A	EX
M		EXM
P		EXMP
.	E	XMP
.	X	MP
.	M	P
.	P	
.		<erro – vazia>

Operações sobre uma fila

- Definida uma fila F: possíveis operações
  - iniciar a fila: init(F)
  - inserir elemento na fila: enqueue(F, elemento)
  - retirar elemento da fila: dequeue(F)
  - primeiro elemento da fila: first(F) - retorna primeiro elemento mas não o remove
  - último elemento da fila: last(F) - retorna último elemento mas não o remove
  - tamanho da fila: len(F)

TAD Fila

```python
class Fila:
    # inicia a fila - lista vazia
    def __init__(self):
        self.items = []
    # devolve True se fila está vazia e False senão
    def is_empty(self):
        return self.items == []
    # enfila novo elemento no final da fila
    def enqueue(self,item):
        self.items.append(item)
    # retira da fila o primeiro elemento
    def dequeue(self):
       if self.is_empty(): return None
        return self.items.pop(0)
    # devolve o valor do elemento do primeiro mas não remove
    def first(self):
       if self.is_empty(): return None
        return self.items[0]
    # devolve o valor do elemento do último mas não remove
    def last(self):
        if self.is_empty(): return None
        return self.items[-1]
    def size(self):
        return len(self.items)
```

TAD Fila

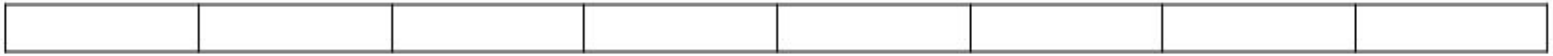
```
# testes da classe Fila
f = Fila()
# enfila 1, 3 e 5 nesta ordem
f.enqueue(1)
f.enqueue(3)
f.enqueue(5)
# mostra início e fim da fila
print("inicio:", f.first(), "fim:", f.last())
# remove e mostra os dois primeiros 1 e 3
print("desenfila:", f.dequeue())
print("desenfila:", f.dequeue())
# mostra início e fim da fila
print("inicio:", f.first(), "fim:", f.last())
```

TAD Fila

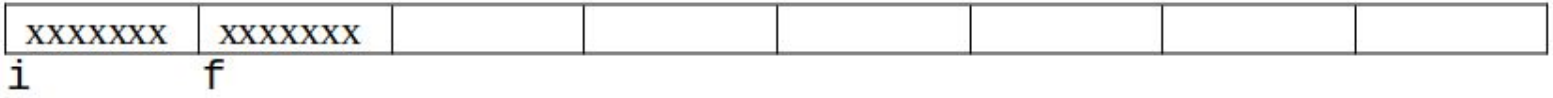
- Implementação alternativa (evitar alocação dinâmica com append() e pop()):
  - Fixar um tamanho máximo para a fila.
  - Controlar o início e fim com índices ou apontadores para elementos da fila.
  - Tornar a lista circular para reaproveitar os elementos que forem sendo esvaziados.
- O uso de uma fila restrita a um tamanho máximo: necessidade em muitas aplicações para controlar o crescimento da fila e a capacidade de processamento dos seus elementos.
- Utilizar 2 apontadores:
  - início da fila (primeiro elemento)
  - fim da fila (último elemento).

TAD Fila

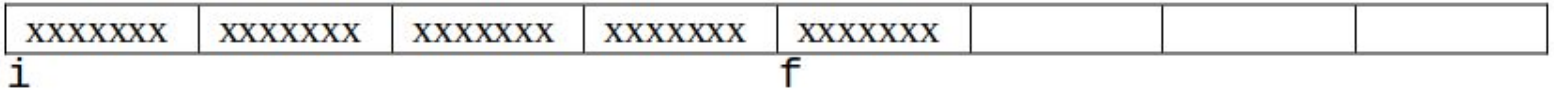
Fila vazia: `i=-1; f=-1;`



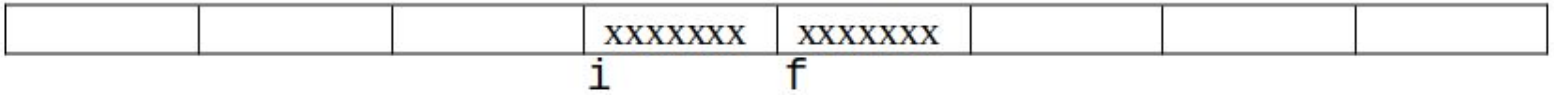
Inserir 2 elementos: `i=0; f=1;`



Inserir mais 3 elementos: `i=0; f=4;`

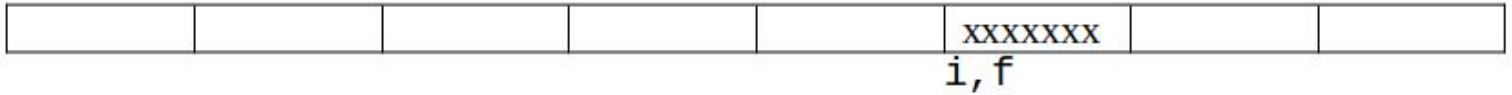


Remover 3 elementos: `i=3; f=4;`

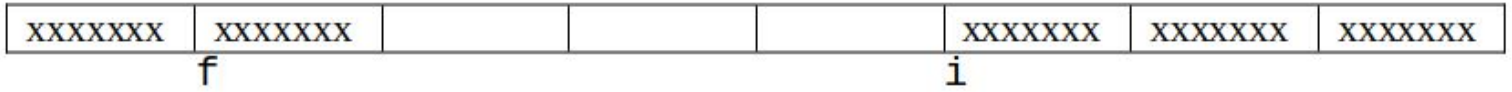


TAD Fila

Inserir 1 elemento e remover 2: `i=5; f=5;`



Inserir 4 elementos: neste caso temos um problema, pois não cabem mais elementos no vetor e temos elementos vazios no início. A fila é circular, isto é, o último elemento do é seguido do primeiro. Ou seja, basta somar um (módulo o tamanho máximo). Neste exemplo teríamos: `i=5; f=1;`



Remover 5 elementos: neste caso também temos um problema, pois a fila ficaria vazia e qual seria o valor de `i` e `f`? Se deixarmos `i=f=2`, por exemplo, não conseguiremos distinguir fila vazia de fila com um só elemento. É melhor neste caso forçar `i=f=-1`.



TAD Fila (2a implementação)

```
class FilaLista:
    # MAXF define o tamanho máximo da fila.
    MAXF = 3 # teste com fila de 3 elementos
    def __init__(self):
        self._fila = [None] * FilaLista.MAXF
        self._size = 0
        self._front = -1
        self._end = -1

    # Adiciona um elemento no final da fila
    def enqueue(self, x):
        # verifica se a fila está cheia
        if self._size == FilaLista.MAXF: return False
        self._end = (self._end + 1) % FilaLista.MAXF
        self._fila[self._end] = x
        # se é o primeiro elemento altera também o inicio
        if self.is_empty(): self._front = 0
        self._size = self._size + 1
        return True
```

TAD Fila (2a implementação)

```
    # Remove um elemento do fim da fila
    def dequeue(self):
        if self.is_empty(): return None
        x = self._fila[self._front]
        self._fila[self._front] = None
        self._size = self._size - 1
        # verifica se a fila vai ficar vazia
        # neste caso, ambos os ponteiros ficam -1
        if self._front == self._end:
            self._front = self._end = -1
        else:
          self._front = (self._front + 1) %
FilaLista.MAXF
        return x
```

```
# Retorna True se a fila está vazia
    def is_empty(self):
        return self._size == 0
    # Devolve o primeiro elemento da fila sem removê-lo
    def first(self):
        if self.is_empty: return None
        return self._fila[self._front]
    # Retorna o tamanho atual da fila
    def __len__(self):
        return self._size
     # Mostra o conteúdo da fila - apenas para teste
    def print(self):
        print(self._fila)
```

TAD Fila (2a implementação)

```
# teste das funções - fila de 3 elementos
F = FilaLista()
# adicionar 4 elementos
for k in [32, 45, 12, 27]:
   if F.enqueue(k): print(k, " adicionado com sucesso")
   else: print(k, "não foi adicionado - fila cheia")
   F.print()
# retirar 2 elementos
print()
print(F.dequeue(), "foi removido")
F.print()
print(F.dequeue(), "foi removido")
F.print()
# primeiro elemento e tamanho
print()
print(F.first(), "é o primeiro elemento da fila")
print(len(F), "é o tamanho atual da fila")
# adicionar 1 elemento
print()
if F.enqueue(-1): print(-1, " adicionado com sucesso")
else: print(-1, "não foi adicionado - fila cheia")
F.print()
```

TAD Fila (2a implementação) - teste de sanidade

```
# retirar 3 elementos
print()
k = F.dequeue()
if k is not None: print(k, "foi removido")
else: print("fila vazia - nada foi removido")
F.print()

k = F.dequeue()
if k is not None: print(k, "foi removido")
else: print("fila vazia - nada foi removido")
F.print()

k = F.dequeue()
if k is not None: print(k, "foi removido")
else: print("fila vazia - nada foi removido")
F.print()
```

TAD Fila

- Algoritmos sequenciais não precisam de filas: melhores exemplos de filas são para processos assíncronos (com velocidades diferentes) compartilhando dados
  - Exemplo: um processo que cria mensagens e outro que processa essas mensagens, em velocidades diferentes. Mantém-se uma fila com as mensagens para processá-las na mesma ordem em que foram criadas
- Extensão da classe FilaLista:
  - Jogar uma moeda (vai simular o tempo de processamento): se der cara, adicionamos um elemento na fila e se der coroa removemos um elemento (usaremos o módulo random)

TAD Fila

```
MAXF = 5 # teste com fila de 5 elementos

# Mostra o conteúdo da fila
def print(self):
    # mostra a fila
    print("* * * status da fila * * *")
    for k in range(FilaLista.MAXF):
        if self._fila[k] is None: print('Vazio', end = ' | ')
        else: print("%05d" %self._fila[k], end = ' | ')
    print()
    # mostra os índices
    for k in range(FilaLista.MAXF): print('%-8d' %k, end = '')
    print()
    # mostra o início e o fim
    print("i =", self._front, " f =", self._end)
    print()
```

TAD Fila

- Supondo a classe FilaLista está dentro do arquivo ClasseFilaLista.py:

```
from ClasseFilaLista import FilaLista
from random import randrange
```

TAD Fila

```
# cria uma fila
NF = FilaLista()
# operações sobre a fila
while True:
    k = randrange(2)
    if k == 0:
        # adiciona elemento
        x = randrange(100000)
        if NF.enqueue(x):
            print(x, "adicionado a fila")
        else:
            print(x, "não foi adicionado - fila cheia")
    else:
        # remove elemento
        x = NF.dequeue()
        if x is not None:
            print(x, "- removido com sucesso")
        else:
            print("nada foi removido - fila vazia")
    # mostra a fila
    NF.print()
    # espera um tempo
    entrada = input("")
```

Observações sobre a implementação do TAD Fila

- Fila cheia e fila vazia:
  - É um erro de programação!
  - Ao invés de retornar um aviso dessa situação a função poderia sinalizar uma exceção.

```
class Empty(Exception):
   # Nova excessão - sub classe de Exception
   pass

class Full(Exception):
   # Nova excessão - sub classe de Exception
   pass

if self.is_empty(): raise Empty("Fila vazia")
if self._size == FilaLista.MAXF: raise Full("Fila cheia")
```

Observações sobre a implementação do TAD Fila

- O estouro do tamanho da fila pode ser resolvido redimensionando-a quando necessário: quando ocorre a condição de fila cheia, redimensiona-se para o dobro do tamanho.

```
def _resize(self, newsize):
  fila_antiga = self._fila # guarda a fila antiga
  self._fila = [None] * newsize
  # reinsere os elementos na fila_antiga para self_fila
.
.
.
if self._size == FilaLista.MAXF:
   self._resize(2*len(self._fila))
```

Observações sobre a implementação do TAD Fila

- Podemos também controlar o tamanho da fila para para não ficar com um tamanho muito grande, com espaços ociosos.

```
if 0 < self._size < len(self._fila) // 4:
  self._resize(len(self._data) // 2)
```

TAD Fila

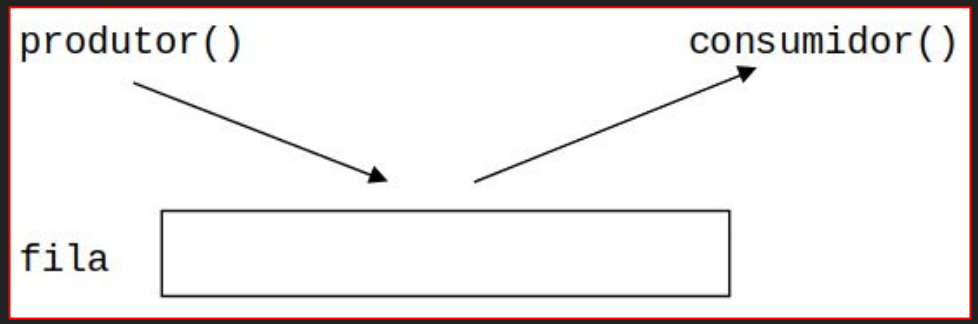
Exercício 1: Criar a classe FilaDupla que admite inserções e remoções no início e no final da fila.

Aplicações de Fila

- O controlador do disco é um processador dedicado para controlar o acesso ao disco:
  - Vários usuários podem requisitar acesso a determinados setores do disco ao mesmo tempo. Como o atendimento a cada requisição demora alguns milisegundos, o controlador tem que gerenciar a fila de requisições dos usuários.
- Um servidor de páginas web que atende um usuário de cada vez recebe ao mesmo tempo várias requisições de páginas de vários clientes da rede:
  - As várias requisições pendentes têm que permanecer numa fila gerenciada pelo protocolo TCP que está em contato com a rede, recebendo as requisições dos usuários.
- Uma parte importante do núcleo do sistema operacional é o escalonamento de processos:
  - Os vários processos se alternam na execução, pois a CPU é uma só. O sistema operacional deve gerenciar a fila de processos prontos, esperando a disponibilidade da CPU.
    - Caso um pouco mais complexo, pois além de sua posição na fila, existe também a prioridade do processo. Isso é implementado pelo sistema operacional com uma fila de prioridades ou com várias filas, uma para cada prioridade.
- A fila da impressora:
  - Em um ambiente em rede a impressora é usada por vários usuários que podem enviar arquivos à impressora para serem impressos. Os arquivos aguardam a impressão numa fila, pois a velocidade de impressão é muito menor que a velocidade de recepção dos arquivos.

Processos Concorrentes

- Filas são usadas, em geral, quando o processamento é feito por processos assíncronos funcionando concorrentemente.
  - A fila regula a comunicação entre os processos.
- Existem os processos “produtores” de elementos para a fila e os processos “consumidores” de elementos da fila.
  - A produção e consumo de elementos da fila ocorre em velocidades diferentes. Daí a necessidade da fila.




```
# processo produtor
def Produtor():
 while True:
    produza (msg)
    # se a fila está cheia, fica esperando
    insere_msg_na_fila(msg);

# processo consumidor
def Consumidor():
 while True:
    # se fila está vazia fica esperando
    remove_msg_da_fila(msg)
    consuma(msg)
```

Processos Concorrentes

Exercício 2: Implementar o problema do produtor e consumidor utilizando conceitos de sincronização do Python.

- Roteiro:
  - Uma **Thread** em Python é um processo em execução ou uma linha de execução de um programa. A classe que contém um programa concorrente é uma classe “filha” da classe Thread e precisa de uma função de nome run.
  - A classe **Condition** especifica os elementos de sincronização entre os processos concorrentes. O sincronismo é implementado através dos métodos:
    - **acquire()** e **release()**: Bloqueia e desbloqueia um trecho de programa ou uma região crítica onde apenas um dos processos pode entrar por vez.
    - **wait()**: Coloca o processo em espera de uma notificação. Só pode ser emitida se o processo está bloqueado. Libera o bloqueio e coloca o processo em espera da notificação. Quando recebe a notificação o bloqueio é retomado.
    - **notify()**: Envia uma notificação a qualquer processo que esteja esperando esta notificação. Não libera o bloqueio imediatamente. O bloqueio só é liberado quando esse processo emitir um release().
    - **<classe>().start()**: Lança a Thread para execução (método run da Thread) e continua a execução normal do programa.

Processos Concorrentes

- Exemplo:
  - Lançamento de duas Threads
  - Proteção da região crítica
  - Simulam processo de produtor e consumidor (a fila vai receber mensagem produzida pelo produtor ou vai ter uma mensagem retirada, a ser consumida pelo consumidor).

```
from threading import Thread, Condition
import time
c = Condition() # para proteger a região crítica
cont = 0 # contador global aos dois programas abaixo
class Prog1(Thread):
    def run(self):
        global cont
        while True:
            c.acquire() # bloqueia região crítica
            cont += 1
            print("msg do Prog1 - ", cont)
            c.release() # libera região crítica
            time.sleep(0.1)
class Prog2(Thread):
    def run(self):
        global cont
        while True:
            c.acquire() # bloqueia região crítica
            cont += 1
            print("msg do Prog2 - ", cont)
            c.release() # libera região crítica
            time.sleep(0.1)
Prog1().start() # Lança o run do Prog1
Prog2().start() # Lança o run do Prog1
```

Fila de prioridades

- Cenário real: filas de prioridades:
  - Atendimento prioritário
  - Tipo de atendimento/prioridade
- Soluções:
  - Várias filas: uma para cada prioridade
    - Ao retirar um elemento: sempre da fila de maior prioridade que possui elementos a serem retirados.
  - Uma fila ordenada por prioridades:
    - Quando se coloca um elemento na fila, não se coloca no final e sim no lugar correspondente à sua prioridade
      - Quebra o conceito de fila, pois não estamos usando o princípio de inserir no final e retirar do início.
- Problemas: elementos com baixa prioridade demoram para ser atendidos (talvez nunca sejam)
  - A administração de filas de prioridades requer algoritmos mais sofisticados.



Obrigado!

Contato:

- la.souza@inf.ufes.br
- github.com/luisouza
- https://scholar.google.com.br/citations?user=3gMIEPYAAAAJ&hl=pt-BR
- https://www.researchgate.net/profile/Luis-Souza-Jr